

THE UNIVERSITY OF CHICAGO
DATA SCIENCE INSTITUTE

Introduction

Kyle Chard, Jonier Amaral Antunes, Alok Kamatar

Welcome!

```
github.com/uchicago-dsi/national-lab-agentic-training
```



Audience

- ➔ Researchers and scientists exploring AI-driven or autonomous workflows
- ➔ Anyone interested in how agentic systems can scale and accelerate discovery

Requirements

- ➔ Familiarity with Python
- ➔ MacOS/Linux/WSL with Python 3.11–3.14

UV

```
$ uv sync
```


Welcome!
National Labs Agentic
Training Workshop

Agents are changing science

Physics & Mathematics > Mathematics

Nobel Prize-winning physicist and team use Claude AI to solve decades old math puzzle

A decade after uncovering a mysterious mathematical relationship in the physics of "jamming," Nobel laureate Giorgio Parisi and collaborator Francesco Zamponi have finally cracked the case — not with a radical new theory, but with the help of the generative AI

October 6, 2026 Research Publication

Sharing AI progress in mathematics

View on GitHub ↗

Listen to article 2:04 Share

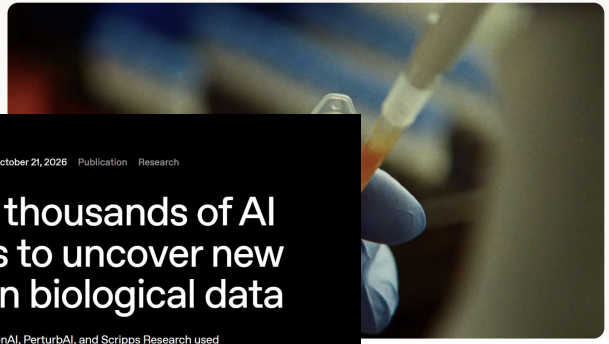
We're releasing a broad range of new mathematical results produced by an internal frontier model.

As we look to improve how we share results with the math community, we've been consulting with the independent Advisory Group on Mathematics and Artificial Intelligence at the Institute for Advanced Study to develop best practices, and we have drawn on their advice and

Science

Claude discovers a novel enzyme system with CRISPR-like repeats

Sep 23, 2026



September 8, 2026 Research Publication

On the Navier–Stoke Millennium Prize Prob

Read the paper Link to Lean formalized proof ›

Listen to article 8:56 Share

We're sharing a solution to the Navier–Stokes existence and smoothness problem, one of the Millennium Prize Problems. This proof, produced by an internal OpenAI system, shows that the dynamics of the Navier-Stokes equations for fluid motion can develop a singularity in finite time. We're sharing both a writeup of the proof and a formalization in Lean.

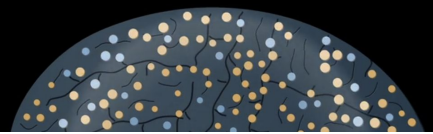
October 21, 2026 Publication Research

ng thousands of AI nts to uncover new ls in biological data

How OpenAI, PerturbAI, and Scripps Research used tributed AI agents to help scientists identify promising research leads in data from millions of brain cells

Read the paper ↗

urbAI



THE UNIVERSITY OF CHICAGO
DATA SCIENCE INSTITUTE

In this course you will

- ➔ Learn what agents are, when they’re the right tool, and how they plug into the scientific lifecycle
- ➔ Build a tool-using agent from scratch and use MCP, retrieval augmented generation, and memory
- ➔ Evaluate agents using harnesses and test datasets
- ➔ Identify and defend against failure modes and security risks
- ➔ Scale agents from one to many agents
- ➔ Consider approaches for observability and cost management
- ➔ Apply agents to your own research problem

People



Kyle Chard
Research Associate Professor
Computer Science
UChicago



Jonier Amaral Antunes
Preceptor
Data Science Institute
UChicago

Course outline

November: How Agents Act

- Go from a single LLM call to a tool-using agent with MCP
- Learn prompting and reasoning fundamentals: system prompts, structured outputs, chain of thought
- Learn how agents handle knowledge through context engineering and retrieval-augmented generation
- Consider how memory can enable recall across interactions

December: How Agents Improve

- Evaluate agents: break them, catalog failure modes, and build a test harness
- Improve systematically, tracking a baseline score, and keeping provenance logs & reasoning traces for reproducibility
- Cover security and safety, including defense with sandboxing and guardrails
- Explore multi-agent patterns, scaling, observability and cost management
- Apply agents to your scientific problems

Pilot training program

- Goal: learn the foundations of agentic systems and leave ready to apply them to your own science
- Session 1:
  a. Day 1 builds the foundations: slower pace with focus on the building blocks
  b. Day 2 builds a coding agent, and we use coding agents from then on, so the pace will pick up quickly
- Session 2: more advanced with application to your problems
- Please give feedback as we go. We're learning too, and we'll adapt the course along the way

Day 1 agenda

Time	Topic
10:00 - 10:45	**Introduction**
10:45 - 11:00	Activity 1: Claim extraction
11:00 - 11:30	**How Agents Think**
11:30 - 12:00	Activity 2: Tree of thought critique
12:00 - 1:00	Lunch
1:00 - 1:30	**How Agents Act: Tools and Frameworks**
1:30 - 2:00	Activities 3-4: Tool calling and LangGraph
2:00 - 2:30	**How Agents Act: Model Context Protocol (MCP) and HPC**
2:30 - 3:00	Activity 5: MCP and HPC tool calling

Important links

Argonne NATIONAL LABORATORY

https://www.alcf.anl.gov/support-center/get-started/request-account

- ANL-FermiAgenticTraining
- PI: Kyle Chard

GitHub

https://github.com/uchicago-dsi/national-lab-agentic-training

slack

https://join.slack.com/t/uchicagoagent-yxv9237/shared_invite/zt-4etfppp03-1F7Jaj8MSRmuPLn8gLRbsQ

THE UNIVERSITY OF CHICAGO DATA SCIENCE INSTITUTE

Level setting

- Who has used an LLM chat interface?
- Who has access to an inference API?
- Who has connected to the ALCF Inference service?
- Who has installed a coding agent?
  - Which coding agent?
  - How often do you use a coding agent?

Questions?

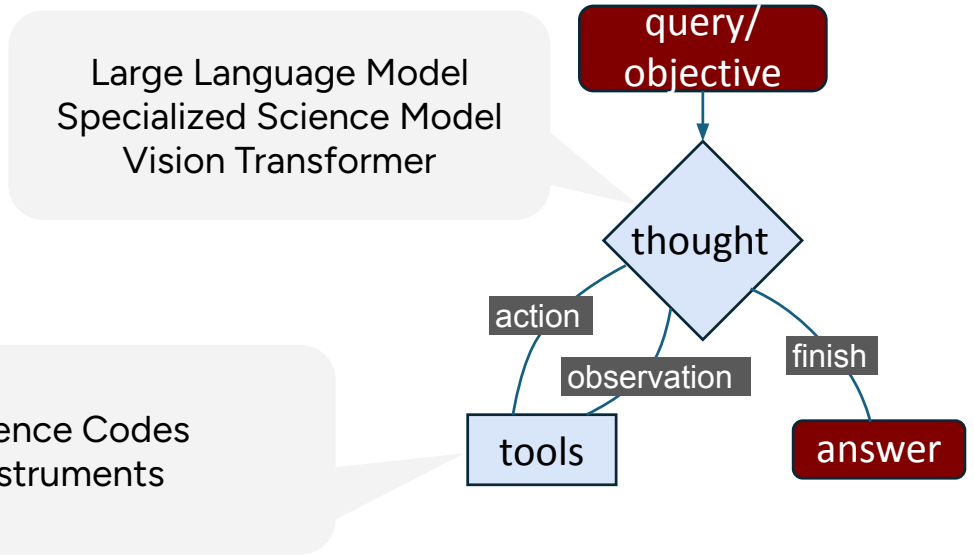
Timeout: pilot program survey

https://tinyurl.com/national-labs-survey

What is an “agent”?

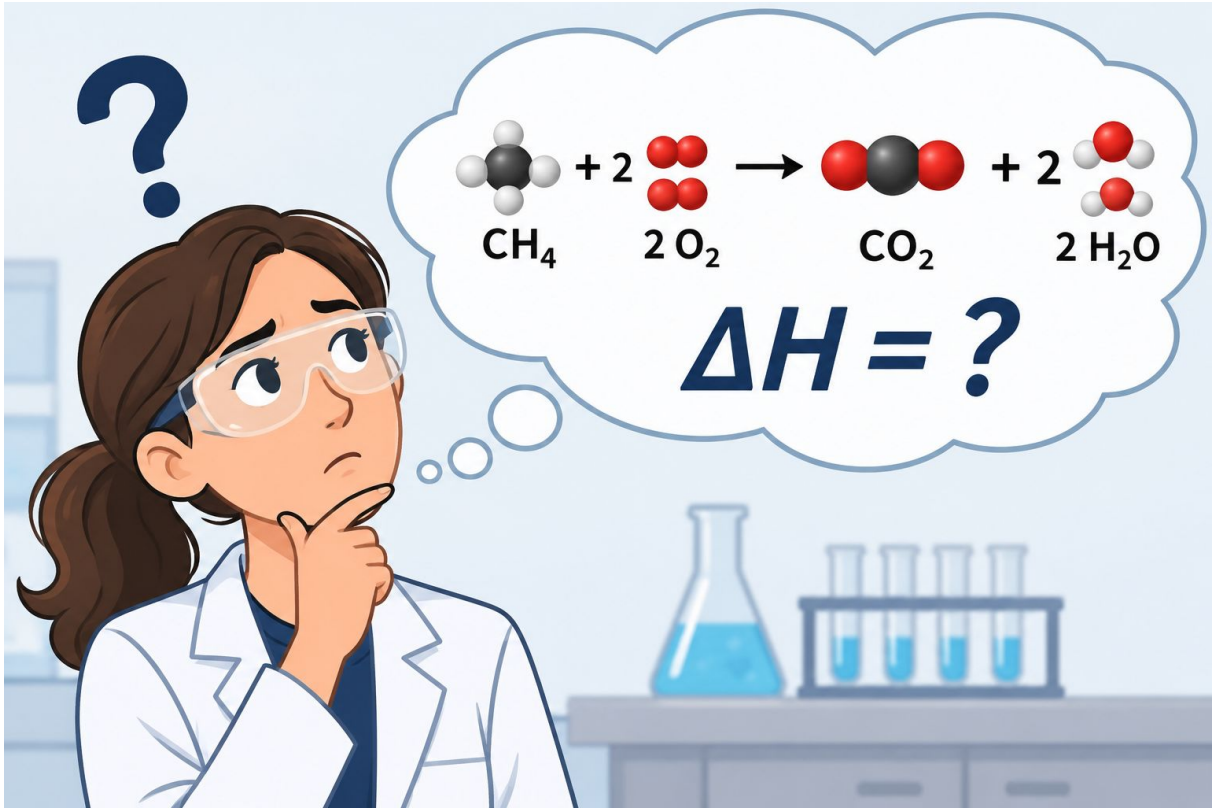
An agent is a persistent, stateful process that acts on behalf of a user or system. An agent may:

- Sense/Observe inputs or events
- Plan (decide on) actions using a policy (rules or LLM)
- Act: Execute tools or call agents
- Learn: Update state to ada over time

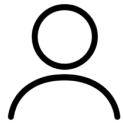


We can think of an agent as a capable colleague that can reason, act, and coordinate autonomously toward an objective

Example: From LLMs to Agents

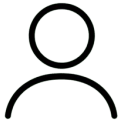


You are given a chemical reaction: 1 (Methane) + 2 (Oxygen) -> 1 (Carbon dioxide) + 2 (Water). Calculate the enthalpy change for this reaction using at 400K



THE UNIVERSITY OF CHICAGO
DATA SCIENCE
INSTITUTE

You are given a chemical reaction: 1 (Methane) + 2 (Oxygen) -> 1 (Carbon dioxide) + 2 (Water). Calculate the enthalpy change for this reaction at 400K

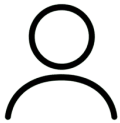




LLM

The reaction enthalpy at 400 K is approximately **-8.3 eV** ✘

You are given a chemical reaction: 1 (Methane) + 2 (Oxygen) -> 1 (Carbon dioxide) + 2 (Water). Calculate the enthalpy change for this reaction **using GFN2-xTB** at 400K



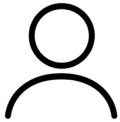


LLM

To actually calculate the enthalpy change **using GFN2-xTB at 400 K**, I would need to run xTB on molecular structures for CH_4, O_2, CO_2, and H_2O and obtain their thermal corrections at 400 K. **I don't have the ability to execute xTB calculations directly.**

* Older hosted LLMs had limited agentic/tool capabilities

You are given a chemical reaction: 1 (Methane) + 2 (Oxygen) -> 1 (Carbon dioxide) + 2 (Water). Calculate the enthalpy change for this reaction using GFN2-xTB at 400K





LLM

The enthalpy change for the reaction at 400 K is approximately -801 kJ/mol

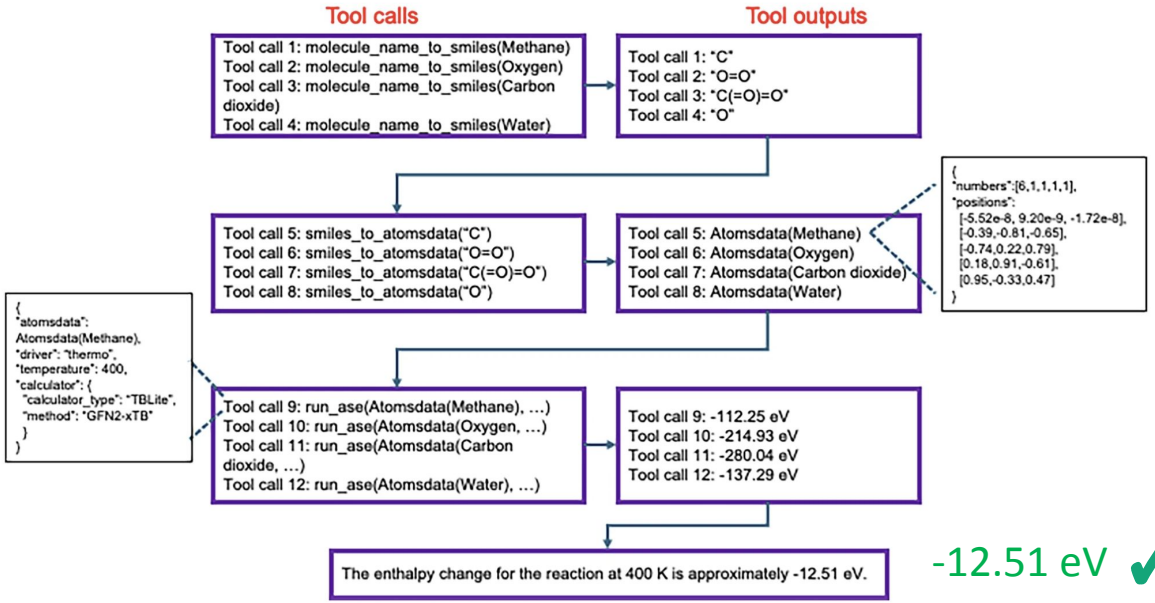
* Newer hosted LLMs have expanded agentic/tool capabilities

You are given a chemical reaction: 1 (Methane) + 2 (Oxygen) -> 1 (Carbon dioxide) + 2 (Water). Calculate the enthalpy change for this reaction using GFN2-xTB at 400K



Murat Keçeli et al.

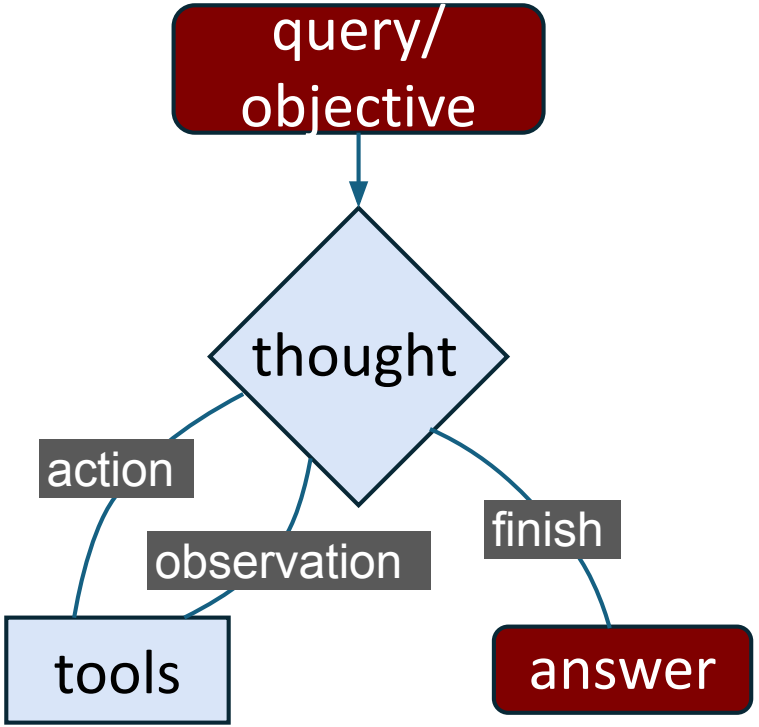
ChemGraph
agent framework:



-12.51 eV ✓

The anatomy of an agent

- Model: decides what to do next; can be an LLM or a specialized science model
- Tools: functions, codes, databases, instruments
- Context and memory: what the model can see now, and what it keeps across sessions
- Harness: runs the loop, executes tool calls, and enforces limits

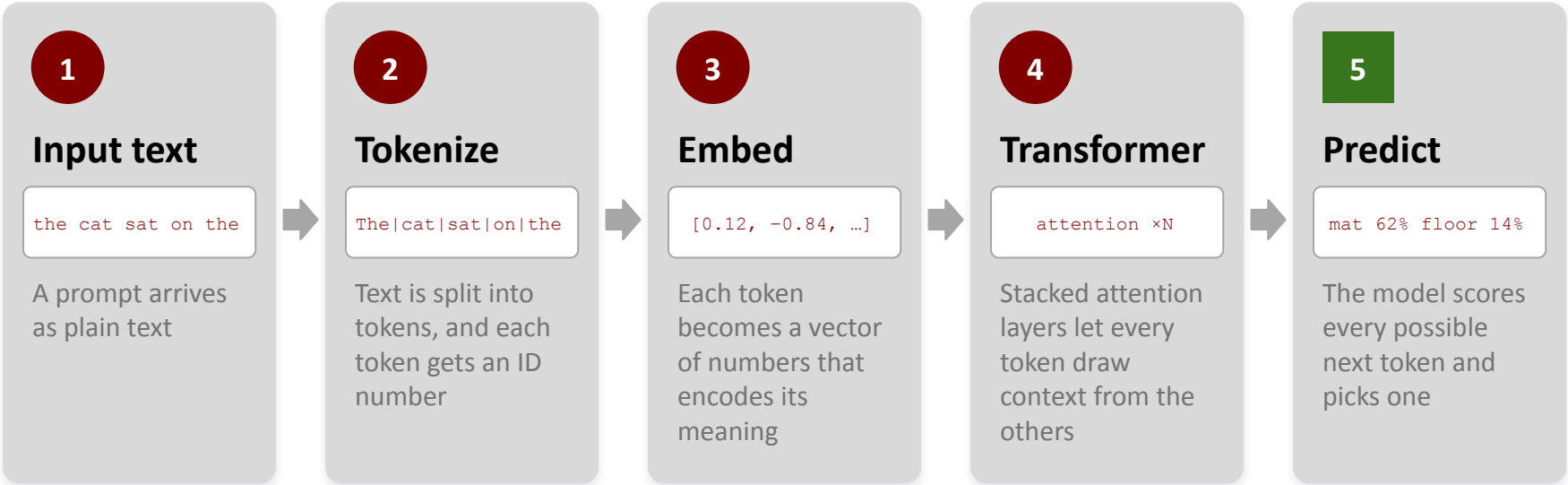


THE UNIVERSITY OF CHICAGO
DATA SCIENCE
INSTITUTE

The anatomy of an agent: **Model**

- Typically an LLM, a neural network trained on huge amounts of text to predict the next token (a word or subword) in a sequence
- Scaling that objective, it learns grammar, facts, reasoning patterns, and code
- It generates output one token at a time (from a distribution), which is why the same prompt can give different answers (temperature controls randomness)
- It sees what's in its context window, and has no built-in memory, no live data, and no ability to act on the world

Detour: How an LLM works



↻ **Repeat:** the chosen token is added to the input, and steps 2–5 run again until the response is complete

The anatomy of an agent: **Tools**

- Agents are “active”:
  - Call tools: read files, query databases, call APIs
  - Execute code: write a script, run it, read output
  - Actuate: control an instrument, steer a robot
- Each tool definition includes its name, description, inputs
- The model decides what tool to use, invokes it, and parses the response to inform the next step

The anatomy of an agent: **Context/Memory**

- Context: what the model sees right now (task, history, tool results)
- Memory: what carries over between sessions (notes, files, databases)
- Both context and memory are limited in size, so deciding what to keep is a design choice

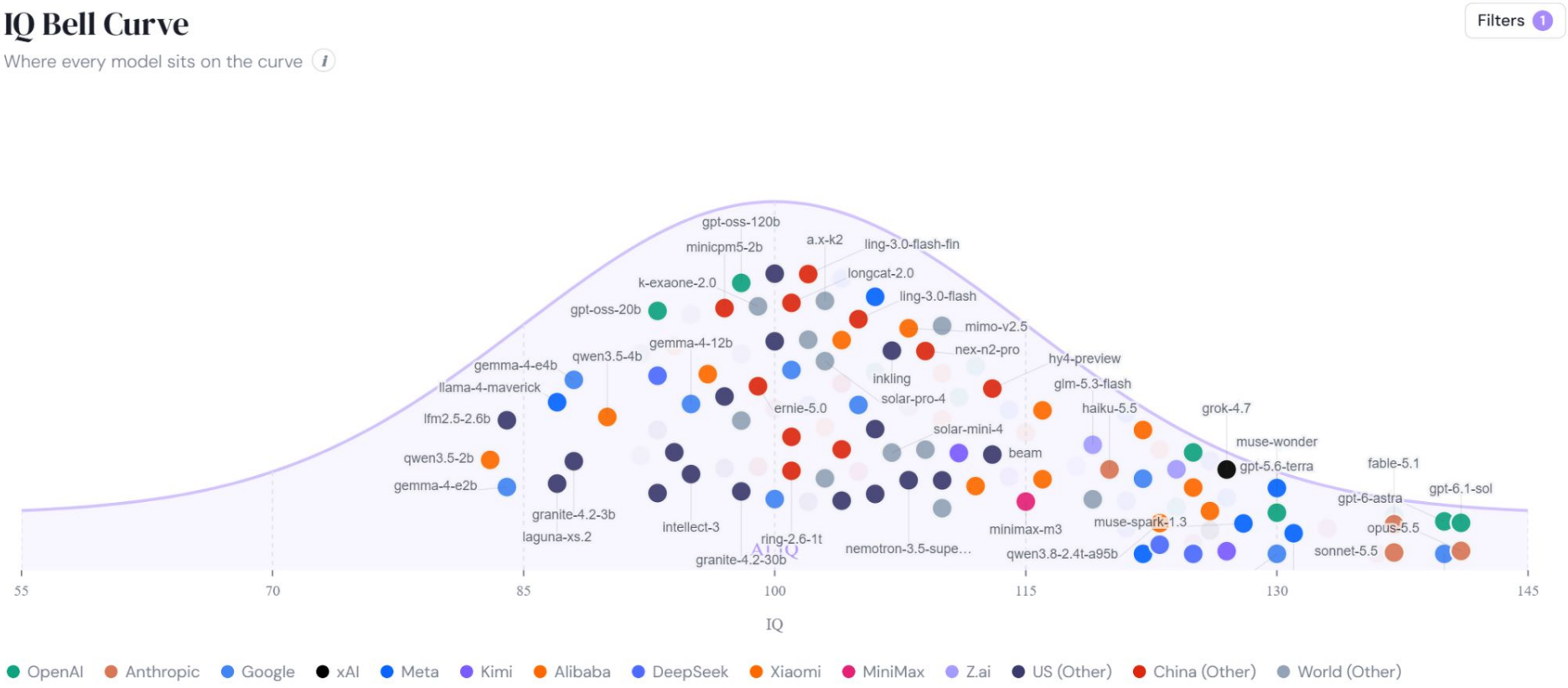
The anatomy of an agent: “**harness**”

The *harness* (sometimes called *app*) is the code wrapped around the model that turns it into an *agent*. The model makes decisions and the harness does everything else

What the harness does:

- Runs the agent loop: send context to the model, read its response, repeat until done
- Executes tool calls on the model’s behalf and feeds results back
- Manages context and memory
- Enforces limits and safety with step caps, token budgets, timeouts, sandboxing, human approval checkpoints
- Logs traces for debugging and provenance

Why are agents so popular?



Why have agents become so popular?



Model capability

Models can now plan multi-step tasks and recover from their mistakes (see previous slide)



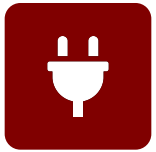
Models trained for tool use

Models are trained to emit structured tool calls and use the results



Long context windows

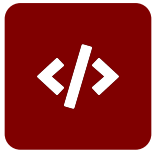
Hundreds of thousands of tokens hold papers, code, and a full history of tool results



Interoperability

MCP provides a common interface to tools, reusable harnesses run the agent loop

Where are agents commonly used?



Coding agents

Read a repository, edit files, and run tests



Research and analysis assistants

Search sources and compile findings



Customer support and workflow automation

Work across email, tickets, and calendars



Computer-use agents

Operate browsers and applications directly

How can agents be used in science?



J. G. Pauloski, K. Chard and I. Foster, "Agentic Discovery: Closing the Loop With Cooperative Agents," in Computer, 2025

Example: Research Assistants

Agents that search, read, and synthesize the literature

Plan
Break the question into searches

Search
Query papers, databases, and the web

Read
Pull out findings from full texts

Synthesize
Write a report with citations

Deep research modes

Claude, ChatGPT, Gemini

Run many searches over several minutes and return a long, cited report on an open question

PaperQA2

FutureHouse

Retrieves and reads full-text papers; reported matching or beating PhD-level experts on literature QA and summarization

Scholarly search tools

Elicit, Consensus, Undermind

Search paper databases directly and extract results into tables for screening and comparison

Example: Co-scientists

Agents that propose hypotheses, plan experiments, and sometimes run them

Coscientist *Boiko et al., Nature 2023*

A GPT-4 agent that planned and ran palladium-catalyzed cross-coupling reactions on automated lab hardware.

ChemCrow *Bran et al., Nature Machine Intelligence 2024*

An LLM with 18 chemistry tools that planned syntheses of an insect repellent and organocatalysts, then ran them on a cloud lab.

AI co-scientist *Google, 2025*

Multi-agent system that generates, critiques, ranks, and refines hypotheses; proposed drug-repurposing candidates for AML later supported in lab tests.

Robin *FutureHouse, 2025*

Multi-agent pipeline that proposed ripasudil as a candidate treatment for dry AMD, with agents designing and analyzing the validation experiments.

Boiko, D.A., MacKnight, R., Kline, B. *et al.* Autonomous chemical research with large language models. *Nature* **624**, 570–578 (2023)
M. Bran, A., Cox, S., Schilter, O. *et al.* Augmenting large language models with chemistry tools. *Nat Mach Intell* **6**, 525–535 (2024)
Gottweis, J., Weng, WH., Daryin, A. *et al.* Accelerating scientific discovery with Co-Scientist. *Nature* **655**, 487–496 (2026)
Ghareeb, A.E., Chang, B., Mitchener, L. *et al.* A multi-agent system for automating scientific discovery. *Nature* **655**, 497–505 (2026)

Examples across the scientific enterprise

Self-driving labs

Agents can drive automated laboratories via structured interfaces (e.g., model hardware standard)

Instrument steering

Tuning beamlines, microscopes, and detectors in real time based on incoming data.

Simulation and HPC

Setting up, launching, and monitoring jobs, as with ChemGraph running xTB earlier.

Scientific software

Coding agents that port, test, and modernize legacy codes, for example for GPUs.

Data curation

Extracting metadata, cleaning datasets, and making data findable and reusable.

Facility operations

Triaging tickets, summarizing logs, and helping schedule shared resources.

Looking forward: dimensions of agency

- Reactive vs. deliberative: responds to each input, or plans several steps ahead
- Single vs. multi-agent: one loop, or many agents coordinating
- Software vs. embodied: works only in software, or acts in the physical world through robots and lab automation
- Autonomy: scripted steps, actions a person approves, fully autonomous runs

Looking forward: where agents fail

- Hallucination: stating false things with confidence, which is especially costly in science
- Tool misuse: calling the wrong tool, passing bad inputs, or ignoring errors
- Getting stuck: looping, drifting off task, or losing sight of the goal during long runs
- Prompt injection: malicious text in a document or webpage that takes over the agent
- Plausible but wrong: a run that completes without errors but uses the wrong setup, such as the wrong spin state, units, or method

Looking forward: cost and control

- Each step of the loop is an LLM call, so cost and latency can add up quickly
- Try to match the model size (cost) to how hard the task is
- Set limits on steps, tokens, and run time
- Human checkpoints must be a deliberate design choice
- More competent agents fail in more complicated ways
- Humans are critical: when many agents explore a problem, their results are narrowed to a short list for human review

Summary

- An agent is an LLM + tools + memory + a harness that runs the loop
- Agents act on the world and learn from the results unlike an LLM that only generates text
- Agents are already useful in science, but people are still needed for judgment and validation
- More capable agents fail in diverse ways, so evaluation and guardrails are essential

Activity 1: Claims Extraction

1. Pick a paper you know well
2. Use an LLM API to extract the paper's major claims
3. Run the same prompt with two models: beam (default) and gpt-oss-120b (on Sophia not Minerva)
4. Compare the results. Discuss with your neighbor.
   a. Which claims did each model capture or miss?
   b. Did either one invent or misstate a claim?

Connecting to a model

ALCF Inference Service

```
$ uv tool install "alcf-ai==0.10.0"
$ alcf-ai auth login
```

Login using your ALCF account

```
$ export ALCF_AI_TOKEN="$(alcf-ai auth get-access-token)"
```

```
$ alcf-ai chat -c minerva -m beam-1.0-bf16  "hello"
```

Configuring OpenAI interface

```
$ export OPENAI_API_KEY="$(alcf-ai auth get-access-token)"
```

```
$ export
OPENAI_BASE_URL=https://inference-api.alcf.anl.gov/resource_server/minerva/api/v1
```

Python to prompt a model

```python
from openai import OpenAI

def main(model: str, abstract: str):
    with open(abstract) as fp:
        text = fp.read()

    client = OpenAI()

    response = client.responses.create(
        model=model,
        input=f'Extract the claims and evidence from this paper:\n {text}',
    )

    print(response.output_text)
```